Amb la presentació d'aquest document els autors afirmen a tots els efectes que tant la pràctica informada com el propi informe és exclusivament fruit del seu treball personal i en cap cas s'ha plagiat total o parcialment.

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola Politècnica Superior d'Enginyeria de Manresa

Weather station system

Activity 2: LoRa-based communication with MQTT integration — Internet of Things Connectivity

Aarón Alcalá Marcos García Jaume Pulido Mikael Rodríguez

October 12, 2026

Contents

1 Introduction **3**

2 LoRa Configuration **3**
2.1 Radio Parameters . . . 3
2.2 Network Compatibility and Interference . . . 4
2.3 Software Design . . . 4

3 LoRa library **4**
3.1 LoRaIOT class (`LoRaIOT.hh` / `LoRaIOT.cpp`) . . . 4
3.2 LoRa parameters (`LoRaConfig.h`) . . . 4
3.3 General configuration (`config.h`) . . . 5

4 TX node (`src/tx/main.cpp`) **5**
4.1 Configuration . . . 5
4.2 Initialisation . . . 5
4.3 Main loop . . . 6

5 RX node **7**
5.1 Main program (`main.cpp`) . . . 7
5.2 MQTT . . . 9
5.3 Frontend . . . 9

6 Experiments **9**
6.1 Transmission power vs. energy consumption and signal strength . . . 9
6.1.1 Test procedure . . . 9
6.1.2 Expected behaviour . . . 10
6.1.3 Results . . . 10
6.1.4 Discussion . . . 10
6.2 Bandwidth and data rate . . . 11
6.3 Quality of Service: packet loss vs. distance . . . 12
6.4 Discussion of the results . . . 12

7 Answers to Guiding Questions **12**

8 Conclusions **13**

1 Introduction

This report documents Activity 2 of the Weather station system, which extends the MQTT chain of Activity 1 by introducing a LoRa link between two ESP32 boards: a transmitter (TX) node that reads the DHT11 sensor and sends the data over LoRa, and a receiver (RX) node that forwards what it receives to the Mosquitto broker over MQTT, so that the dashboard remains unchanged.

For this activity, two groups joined forces: Aarón Alcalá and Marcos García on one side, and Jaume Pulido and Mikael Rodríguez on the other. The work was split so that each group programmed one of the two nodes. Jaume and Mikael's group developed the TX node, including the handling of the DHT11 sensor, and Aarón and Marcos's group developed the RX node, where the WiFi connection, the MQTT publication to the Mosquitto broker and the web frontend keep the same idea and structure as in Activity 1.

2 LoRa Configuration

The LoRa communication system enables two ESP32-based boards to exchange sensor information wirelessly. The objective is to provide a long-range communication link suitable for IoT applications while keeping the implementation modular and easy to maintain.

The configuration and communication logic are separated into different files. The `LoRaConfig.h` file defines the radio configuration structure, while `config.h` contains the default radio parameters and hardware pin assignments. The `LoRa_IOT.h` and `LoRa_IOT.cpp` files define the communication interface and its implementation. This separation allows the radio parameters to be modified without changing the application logic.

2.1 Radio Parameters

The radio configuration is centralised in `config.h` and passed to the initialisation function through the `LoRaConfig` structure. The main parameters and their design rationale are described below.

- **Frequency (868 MHz):** Selected for operation in the European 868 MHz band, subject to applicable regulations. Both boards must use the same frequency to communicate.
- **Bandwidth (250 kHz):** Provides a relatively high data rate compared with narrower bandwidths, making it suitable for transmitting small sensor messages periodically.
- **Spreading factor (SF7):** Selected to favour a higher data rate and shorter transmission times. This is suitable for small sensor messages, provided that the required communication range and signal quality are maintained.
- **Coding rate (4/5):** Adds forward error correction to improve resilience against transmission errors while limiting additional transmission overhead.
- **Sync word (0x12):** Helps distinguish our LoRa network from networks using different sync words. Both boards must use the same value.
- **Preamble length (8 symbols):** Supports packet detection and synchronisation while limiting transmission overhead.

- **CRC enabled:** Allows the receiver to detect corrupted packets and identify yransmission errors.

The transmit power is also configurable. Its final value must bec ompatible with the radio module and comply with the applicable regional regulations.

2.2 Network Compatibility and Interference

Reliable communication requires both ESP32 boards to use compatible radio parameters, including frequency, bandwidth, spreading factor, coding rate, and sync word. Centralising these values ensures consistency and simplifies configuration changes.

The selected sync word helps distinguish our network from other LoRa networks using different settings. However, LoRa operates over a shared radio channel, meaning that other devices transmitting on the same frequency may still cause interference or packet collisions. Therefore, this configuration reduces the likelihood of accepting packets from unrelated networks but cannot guarantee exclusive communication between our two boards.

Furthermore, the sync word does not provide encryption or sender authentication. Additional security mechanisms would be required if confidentiality or protection against unauthorised transmissions were necessary.

2.3 Software Design

The communication software follows a modular design, separating radio configuration from the application that processes sensor data. The public interface provides functions for initialisation, transmission, and reception, while the implementation manages the radio hardware through the Arduino LoRa library.

This approach simplifies integration with the rest of the IoT application and allows the communication layer to be maintained independently. A GPIO interrupt is also configured to signal radio events, keeping the interrupt handler lightweight. In the current implementation, packet reception is handled through polling, while the interrupt flag is not yet integrated into the reception logic.

Overall, the selected configuration establishes a common radio profile for both boards, while the modular software structure facilitates maintenance and future adaptation. The final parameters should be validated under the expected operating distances and environmental conditions.

3 LoRa library

⟨Introducción breve: por qué se encapsula LoRa en una librería propia (`LoRaIOT`) y qué problema resuelve.⟩

3.1 LoRaIOT class (`LoRaIOT.hh` / `LoRaIOT.cpp`)

⟨Interfaz pública, funciones principales (inicialización, envío, recepción), librería base usada.⟩

3.2 LoRa parameters (`LoRaConfig.h`)

⟨Parámetros configurables: frecuencia, spreading factor, bandwidth, coding rate, potencia de transmisión, sync word. . . ⟩

3.3 General configuration (`config.h`)

⟨Pines del módulo LoRa, parámetros comunes entre TX y RX, intervalo de envío.⟩

4 TX node (`src/tx/main.cpp`)

The transmitter is the node placed at the weather station. Its only task is to read the DHT11 sensor periodically and send each reading over LoRa to the receiver, which acts as a gateway towards the MQTT broker. Because it does not need any Internet connectivity, the transmitter does not use WiFi or MQTT at all, which keeps the firmware small and reduces its power consumption.

The program is built with PlatformIO in its own environment (`env:tx`), which compiles only the `src/tx/` folder through `build_src_filter`. The radio code is shared with the receiver and lives in the `lib/lora` library (`LoRa_IOT.h/.cpp`), so the main file only contains the application logic.

4.1 Configuration

All the parameters are defined in `include/config.h` and loaded into a `LoRaConfig` structure (`lib/lora/LoRaConfig.h`), whose default values come from these macros. Both nodes use the same structure, which guarantees that they share the same radio settings. Table 1 summarises the values used.

Parameter	Value
Board / radio	Heltec WiFi LoRa 32 V2 / SX1276
Frequency	868 MHz (EU868)
Bandwidth	250 kHz
Spreading factor	7
Coding rate	4/5
Preamble length	8 symbols
Sync word	`0x12`
TX power (default)	20 dBm
CRC	Enabled
Sensor	DHT11 on GPIO 13
Send interval	5 s (`LORA_SEND_INTERVAL_MS`)

Table 1: LoRa and application parameters used by the transmitter.

4.2 Initialisation

The `setup()` function opens the serial port for debugging, starts the DHT11 driver and initialises the radio with `LoRaIOT_setup()`:

```
DHT dht(DHT_PIN, DHT_TYPE);
LoRaConfig loraConfig; // default values from config.h
int counter = 0; // packet id

void setup(){
    Serial.begin(115200);
```

```
    dht.begin();

    if (!LoRaIOT_setup(loraConfig))
        while (1); // stop if the radio does not answer
}
```

Listing 1: Initialisation of the transmitter.

`LoRaIOT_setup()` starts the SPI bus on the pins of the on-board SX1276, sets the chip-select, reset and DIO0 pins, calls `LoRa.begin()` with the carrier frequency and then applies the spreading factor, bandwidth, coding rate, preamble length, sync word and transmission power, and enables the CRC. If the radio does not respond, the function returns `false` and the program stops, since the node is useless without the radio.

4.3 Main loop

Every iteration of `loop()` performs three steps: it reads the sensor, builds the message and sends it.

```
void loop(){
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity)) {
        Serial.println("Error leyendo el DHT11");
        delay(2000);
        return; // retry without sending anything
    }

    String message = "{\"id\":" + String(counter) + ",\"temperature\":" + String(temperature,
        ↪ 1) + ",\"humidity\":" + String(humidity, 1) + "}";

    if (!LoRaIOT_send(message))
        Serial.println("Error enviando el paquete");

    counter++;
    delay(LORA_SEND_INTERVAL_MS);
}
```

Listing 2: Main loop of the transmitter.

Sensor reading. The DHT11 returns `NaN` when a reading fails (for example, if it is read too often or a wire is loose). In that case the iteration is discarded and repeated after 2 s, so invalid values are never transmitted.

Message format. Each reading is encoded as a small JSON object:

```
{"id":12,"temperature":23.4,"humidity":55.0}
```

Values are rounded to one decimal, which matches the resolution of the DHT11. JSON was chosen because the receiver can parse it directly with ArduinoJson and the format is easy to read in the serial monitor. The field `id` is a counter that increases with every packet; the receiver can use it to detect lost packets by looking for gaps in the sequence. The message is about 43–45 bytes long, far below the 255-byte limit of a LoRa packet.

Transmission. `LoRaIOT_send()` wraps the three calls of the Arduino LoRa library: `beginPacket()`, `print()` and `endPacket()`. The last one is blocking: it returns when the radio signals the end of the transmission (*TxDone*), so when `LoRaIOT_send()` returns the packet has already been sent.

Send period. After each packet the node waits `LORA_SEND_INTERVAL_MS` (5 s). This respects the DHT11 minimum sampling period (1 s) and keeps the channel occupancy low: with a time on air of around 44 ms (see Section 6.1.2), the duty cycle is below 1%, which is the limit of the EU868 sub-band used.

5 RX node

The RX node is the bridge between the LoRa link and the MQTT chain built in Activity 1. Its task is to wait for the packets sent by the TX node, extract the temperature and humidity from them and republish them to the Mosquitto broker, so that the dashboard keeps working exactly as before. The WiFi connection, the MQTT publication and the web frontend follow the same idea and structure as in Activity 1, so the only new element on this node is the LoRa reception.

5.1 Main program (`main.cpp`)

The program is built with PlatformIO in its own environment (`env rx`), which compiles only the `src/rx/` folder through `build_src_filter`.

The `main.cpp` of the RX node reuses the `WiFiManager` and `MqttPublisher` classes of Activity 1 and adds the LoRa functions provided by the library described in Section 3. Besides the WiFi and MQTT objects, the program only needs one extra global variable, `rxFlag`, a `volatile bool` that signals that a LoRa packet has arrived.

Initialization. In `setup()`, the WiFi manager is started as in Activity 1, and the radio is configured through `LoRaIOT _setup(LoRaConfig())`. This is the same call used by the TX node, so both boards share exactly the same radio configuration (frequency, spreading factor, bandwidth, coding rate, preamble length and sync word); otherwise the receiver would not be able to decode the packets. If the radio fails to start, the program stops, since the node would be useless without it. Finally, `LoRa.receive()` is called to put the radio in continuous receive mode. This call is essential: after `LoRa.begin()` the module is not listening by default, so without it no packet would ever be detected and the interrupt described below would never fire.

Interrupt-driven reception. Instead of repeatedly asking the radio whether a packet has arrived, the RX node relies on a hardware interrupt. An interrupt makes the microcontroller run a function only when a given event happens, which here is the arrival of a packet. During `LoRaIOT _setup()`, `attachInterrupt()` associates the DIO0 pin of the LoRa module with the function `onLoRaRxIRQ()`, which is executed on every rising edge of that pin, i.e. when the module signals that a complete packet has been received:

```
1  extern volatile bool rxFlag;
2
3  void IRAM_ATTR onLoRaRxIRQ()
4  {
5      // The interrupt only signals that a packet has arrived
```

```
    rxFlag = true;
}
```

The interrupt routine does nothing but raise the flag, since this kind of routine must be as short as possible. The variable is declared `volatile` in `main.cpp` because it is modified from the interrupt and read from `loop()`, so the compiler must not assume that its value stays unchanged between reads. The library file accesses it through `extern`, and the `IRAM_ATTR` attribute keeps the routine in the ESP32 internal RAM so it can run quickly. The heavy work (reading the packet, parsing it and publishing it) is done later in the main loop, outside the interrupt.

Main loop. On every iteration, `loop()` follows these steps:

1. The WiFi manager is updated, and the iteration is skipped if there is no connection yet.
2. If the MQTT connection has not been established, it is attempted again, and `mqttPublisher.loop()` is called to keep it alive. These steps work exactly as in Activity 1.
3. If `rxFlag` is set, it is cleared first and then the packet is read with `LoRaIOT_receive()`.
4. If a packet was read correctly and the MQTT connection is available, the message is handed to `publishReading()`.

In contrast with Activity 1, the publication is no longer periodic: nothing is sent to the broker until a LoRa message arrives, so the publishing rate is set by the TX node.

Packet reception. The `LoRaIOT_receive()` function checks for an incoming packet using `LoRa.parsePacket()`. If no packet is available, it returns `false`. Otherwise, it reads the packet byte by byte using `LoRa.available()` and `LoRa.read()`, storing the received data in the `message` string. It then prints the message and its RSSI value, which indicates the received signal strength. Finally, `LoRa.receive()` puts the radio back into reception mode, and the function returns `true` to indicate that a packet has been received.

From the LoRa message to MQTT. The message received over LoRa is a JSON string. `publishReading()` deserializes it with **ArduinoJson**, discarding the packet and reporting the error if the parsing fails. Then it extracts only the temperature and the humidity, ignoring the rest of the fields of the received JSON, such as the packet counter `id`, since that information is not going to be sent over MQTT. Finally, it builds a new JSON with the `temp` and `hum` fields through the `MqttPublisher` and publishes it to the broker:

```
StaticJsonDocument<200> doc;
DeserializationError error = deserializeJson(doc, message);

if (error) return false;
float temperature = doc["temperature"];
float humidity = doc["humidity"];

publisher.addField("temp", temperature);
publisher.addField("hum", humidity);
publisher.publish(topic, retained);
```

Rebuilding the JSON with the same fields as in Activity 1 is what allows the rest of the chain to remain unchanged: the broker and the dashboard receive the same message on `sensor/1` as if the sensor were connected directly to the node.

5.2 MQTT

The MQTT part of the RX node is the same as in Activity 1. The `WiFiManager` state machine handles the connection to the WiFi network without blocking, and the `MqttPublisher` class connects to the Mosquitto broker, reconnects automatically if the connection is lost, and serializes and publishes the JSON message. The only difference is that the readings now come from the LoRa packets instead of a local sensor.

5.3 Frontend

The frontend does not change either. `dashboard.py` keeps subscribing to `sensor/+`, storing the readings in bounded queues and drawing the live charts with Dash and Plotly, as described in Activity 1. Since the RX node publishes the same JSON format and topic, the dashboard cannot tell whether the data comes from a sensor wired to the node or from a remote one over LoRa.

6 Experiments

6.1 Transmission power vs. energy consumption and signal strength

The aim of this experiment is to evaluate how the transmission power affects the signal strength at the receiver and the energy that the transmitter needs to send each packet. In a battery-powered sensor node there is a trade-off between both: a higher power gives a stronger and more robust link, but drains the battery faster.

6.1.1 Test procedure

The experiment is automated with a PlatformIO test, `test/test_tx_power/test_tx_power.cpp`, written with the Unity framework. It runs on the transmitter board and is launched with its own environment, `env:tx_power`, which extends `env:tx` and only runs this test:

```
pio test -e tx_power
```

The test performs the following steps:

1. It initialises the radio with `LoRaIOT_setup()` and the same configuration as the normal firmware (Table 1). If this fails, the test fails.
2. For each power level in $\{2, 5, 8, 11, 14, 17, 20\}$ dBm, it sets the power with `LoRa.setTxPower()` and sends 10 packets, 500 ms apart. Each packet carries the power used and a sequence number, so the receiver can tell which level it belongs to:

   ```
   {"test":"tx_power","power":14,"seq":3}
   ```
3. For each packet it measures the time taken by `LoRaIOT_send()`. Since `endPacket()` blocks until *TxDone*, this is the time on air of the packet plus a small SPI overhead.
4. At the end of each level it prints a CSV line with the power, the number of packets sent and failed, and the average time on air, and waits 3 s before the next level.

5. Finally, it restores the default power from `config.h`. The test passes if every packet was transmitted successfully.

The receiver runs its normal firmware (`env:rx`) without any change. For every packet it prints the payload and its RSSI on the serial monitor, so the RSSI of each power level is obtained from the receiver's log.

The power consumption cannot be measured from the firmware, so a USB power meter was placed between the power supply and the transmitter. The current was noted during each power level; the 3 s pauses between levels leave time to read it. The energy spent per packet is then

$$E_{\text{packet}} = V \cdot I_{\text{TX}} \cdot t_{\text{air}}, \tag{1}$$

where V is the supply voltage (5 V over USB), I_{TX} the current measured at that power level and t_{air} the time on air reported by the test.

Setup. **[TODO: Describe the physical setup: distance between the two boards, indoor/outdoor, line of sight or not, antennas, USB meter model. Add a photo if possible.]**

6.1.2 Expected behaviour

The time on air of a LoRa packet only depends on the spreading factor, the bandwidth, the coding rate, the preamble and the payload length, not on the transmission power. With the parameters of Table 1, the symbol time is

$$T_{\text{sym}} = \frac{2^{SF}}{BW} = \frac{2^7}{250\,\text{kHz}} = 0.512\ \text{ms}, \tag{2}$$

and, using the formula from the SX1276 datasheet, the 37–38-byte test packets need 68 payload symbols. Adding the preamble ($8 + 4.25$ symbols), the expected time on air is about

$$t_{\text{air}} = (12.25 + 68) \cdot 0.512\ \text{ms} \approx 41\ \text{ms}. \tag{3}$$

For the 43-byte weather messages of the normal firmware it is about 44 ms. Therefore, the measured time on air should be the same at every power level, and the energy per packet should change only because of the current drawn by the power amplifier. The RSSI is expected to grow roughly 1 dB for every dB of extra transmission power, until the receiver saturates when the boards are very close.

6.1.3 Results

[TODO: Paste the CSV printed by the test and fill in the RSSI (average of the 10 packets of each level, from the receiver log) and the current from the USB meter. Compute the energy with Eq. (1).]

6.1.4 Discussion

Time on air. **[TODO: Check that t_{air} is constant for all the power levels and compare it with the expected 41 ms. Explain any difference (SPI overhead, `millis()` resolution).]**

Power (dBm)	Received (of 10)	t_{air} (ms)	RSSI (dBm)	Current (mA)	Energy/packet (mJ)
2	–	–	–	–	–
5	–	–	–	–	–
8	–	–	–	–	–
11	–	–	–	–	–
14	–	–	–	–	–
17	–	–	–	–	–
20	–	–	–	–	–

Table 2: Results of the transmission power sweep.

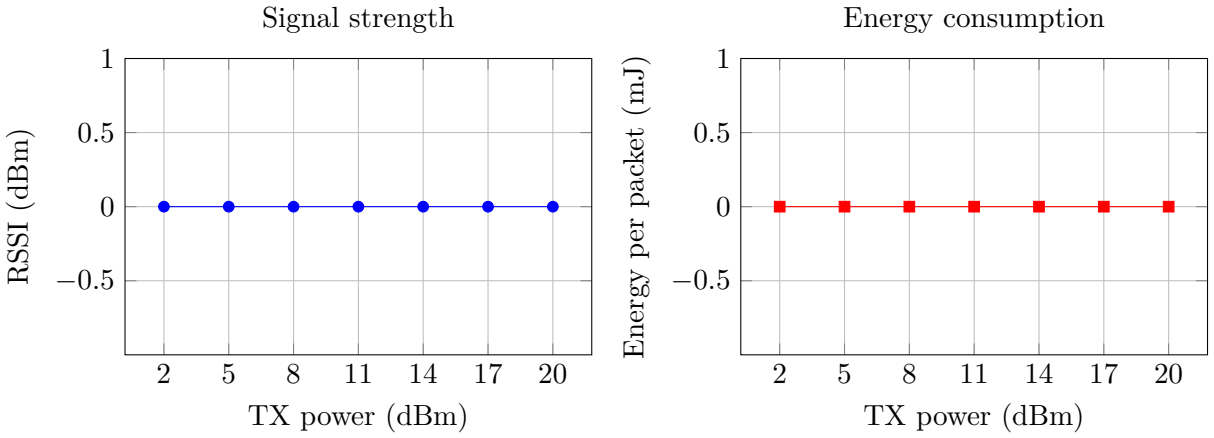


Figure 1: RSSI at the receiver (left) and energy per packet at the transmitter (right) as a function of the transmission power.

Signal strength. **[TODO: Comment on how the RSSI changes with the power. Is the relation linear (about 1 dB per dB)? Does it saturate at high power because the boards are close? Were any packets lost at low power?]**

Energy consumption. **[TODO: Comment on how the current and the energy per packet grow with the power. Which levels are most expensive? How much more energy does 20 dBm need compared with 14 dBm?]**

Trade-off. **[TODO: Conclude which power level is the best choice for this deployment: the lowest one that still gives a reliable link with some margin. Remember that in the EU868 band the legal limit is usually 14 dBm ERP, so 17 and 20 dBm were only used for this lab test.]**

6.2 Bandwidth and data rate

⟨Variación de bandwidth / spreading factor; resultados en tabla o gráfica.⟩

6.3 Quality of Service: packet loss vs. distance

⟨Medida de pérdida de paquetes a distintas distancias; resultados.⟩

6.4 Discussion of the results

⟨Análisis conjunto de los resultados.⟩

7 Answers to Guiding Questions

Q1. How does LoRa compare to Wi-Fi and Bluetooth in terms of range and power consumption?

MARCOS: LoRa trades speed for range and low consumption. It can reach several kilometres while using very little energy, but it can only send small amounts of data slowly. Wi-Fi is much faster, but its range is limited to a building or a house, it needs an access point and it consumes much more energy. Bluetooth also consumes little, but its range is only a few tens of metres. This is why LoRa fits our case: the TX node only sends a small reading every few seconds and does not need any network infrastructure.

MIKA: LoRa provides a significantly greater communication range than Wi-Fi and Bluetooth in many practical IoT deployments, often reaching hundreds of metres or several kilometres under suitable conditions. Wi-Fi and Bluetooth are generally intended for shorter distances, although their actual range depends on the hardware and environment.

LoRa is particularly suitable for transmitting small amounts of sensor data over long distances while allowing devices to operate with low average power consumption. Wi-Fi supports much higher data rates but can consume more energy, while Bluetooth generally offers lower power consumption for short-range communication, although this depends on the Bluetooth version and operating mode.

For our ESP32 project, LoRa is appropriate because the boards only need to exchange small sensor measurements periodically, rather than transmit large amounts of data. This allows long-range communication without requiring the high data rates provided by Wi-Fi.

Q2. What are the trade-offs of using lower data rates in LoRa transmission?

MARCOS: Sending the data more slowly makes the signal easier for the receiver to detect, so the link reaches further and resists noise better. The price is that each message takes longer to send. The radio stays on for more time, so every message costs more energy, there is a higher chance of colliding with other transmissions, and the legal limits on how long a device may transmit allow fewer messages per hour. The data also arrives with more delay.

MIKA: Reducing the LoRa data rate generally increases receiver sensitivity and potential communication range, but also increases packet transmission time. This is commonly achieved by increasing the spreading factor or reducing the bandwidth.

Longer transmission times increase the energy required to send each message and occupy the shared radio channel for longer, increasing the possibility of packet collisions. Therefore, a lower data rate does not necessarily result in lower energy consumption per message.

In our configuration, SF7 and a bandwidth of 250 kHz favour shorter transmission times. This is appropriate for small sensor messages, provided that the receiver can decode them reliably. A higher spreading factor could be used if greater range or better sensitivity were required.

Q3. How does increasing transmission power affect battery life and interference?

MARCOS: A higher transmission power gives a stronger signal, but the radio needs more current to send it, so each message costs more energy and the battery lasts less. In exchange, the extra range is small. A stronger signal also interferes more with other devices using the same frequency band and, if the boards are very close, it can even overload the receiver. For this reason, the best practice is to use the lowest power that still gives a reliable link, which is what the power experiment of Section 6.1 tries to find.

MIKA: Increasing transmission power can improve the received signal strength and extend the potential communication range. However, it also generally increases the energy drawn by the radio during transmission, potentially reducing battery life.

For example, increasing output power from 10 dBm to 17 dBm increases the transmitted radio-frequency power from 10 mW to approximately 50 mW. The battery consumption does not necessarily increase by the same factor, since the radio's efficiency and other power requirements must also be considered.

Higher transmission power can also increase interference for other devices sharing the same or overlapping frequencies. Therefore, the appropriate strategy is to use the minimum power that maintains a reliable connection between the two ESP32 boards.

In our project, RSSI measurements and packet reception tests can help determine whether the selected power provides sufficient reliability without unnecessary energy consumption or interference. The final setting must also comply with the applicable radio regulations.

8 Conclusions

⟨Conclusiones conjuntas.⟩